

Python Data Structures

Lists, Tuples, Sets & Dictionaries

[]

Lists

Mutable, ordered
collections

()

Tuples

Immutable, ordered
collections

{ }

Sets

Mutable collections of
unique elements

{ : }

Dictionaries

Key-value pairs, fast
lookups



Introduction to Python Data Structures

In Python, data structures are fundamental tools for organizing and storing data efficiently.

They provide a structured way to manage collections of information, making it easier to access, access, modify, and process data within your programs.

Key Benefits:

- Efficient data organization
- Optimized data retrieval
- Simplified data manipulation
- Improved program readability

[]

Lists

Dynamic arrays that are ordered and mutable

()

Tuples

Immutable sequences of elements

{ }

Sets

Collections of unique elements

{ : }

Dictionaries

Key-value pairs for fast lookups

Lists: The Dynamic Array

Definition

A versatile, built-in data structure that functions as an ordered, mutable collection of items.

Example

```
my_list = [10, "hello", 3.14, True, 10]
print(my_list)
```

Output: [10, 'hello', 3.14, True, 10]

Key Characteristics



Ordered

Elements maintain their insertion order, with fixed index starting from 0.



Mutable

Elements can be added, removed, or modified after creation.



Allows Duplicates

Lists can contain multiple occurrences of the same element.



Dynamic Sizing

Lists can grow or shrink in size as needed, without predefined size.



Heterogeneous

A single list can hold a mix of different data types (e.g., integers, strings, floats).

List Creation and Access Methods

Creating Lists

Lists are created using square brackets [].

```
# Creating a list
my_list = [10, "hello", 3.14, True, 10]
print(f"Original list: {my_list}")
```

Accessing Elements by Index

Elements can be accessed using positive (0-indexed) or negative indices (-1-indexed).

```
# Accessing elements
print(f"First element: {my_list[0]}")
print(f"Last element: {my_list[-1]}")

# Output:
# First element: 10
# Last element: 10
```

Slicing Lists

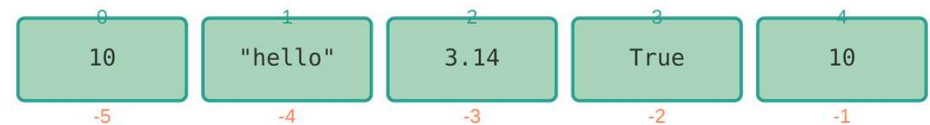
Slicing accesses a range of elements using [start:end], where **end** is exclusive.

```
# Slicing a list

print(f"Elements from index 1 to 3: {my_list[1:4]}")
print(f"Elements from beginning to index 2: {my_list[:2]}")

# Output:
# Elements from index 1 to 3: ['hello', 3.14, True]
# Elements from beginning to index 2: [10, 'hello']
```

Visual Index Reference



Essential List Methods

append(item)

Adds an item to the end of the list.

```
my_list = [1, 2, 3]
my_list.append(4)
print(my_list) # Output: [1, 2, 3, 4]
```

insert(index, item)

Inserts an item at a specified position.

```
my_list = [1, 2, 4]
my_list.insert(2, 3)
print(my_list) # Output: [1, 2, 3, 4]
```

remove(item)

Removes the first occurrence of a specified item.

```
my_list = [1, 2, 3, 2]
my_list.remove(2)
print(my_list) # Output: [1, 3, 2]
```

pop([index])

Removes and returns the item at the given position.
position.

```
my_list = [1, 2, 3]
popped = my_list.pop()
print(popped) # Output: 3
print(my_list) # Output: [1, 2]
```

sort(key=None, reverse=False)

Sorts the items of the list in place.

```
my_list = [3, 1, 4, 1, 5]
my_list.sort()
print(my_list) # Output: [1, 1, 3, 4, 5]
```

len(list)

Returns the number of items in a list.

```
my_list = [1, 2, 3]
length = len(my_list)
print(length) # Output: 3
```

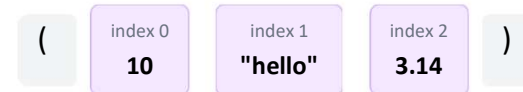
Tuples: The Immutable Sequence

A tuple in Python is an **ordered collection of elements** that is **immutable** (cannot be changed after creation).

Key Characteristics:

- **Ordered** - Elements maintain insertion order
- **Immutable** - Cannot be modified after creation
- **Allows Duplicates** - Can contain duplicate elements
- **Heterogeneous** - Can contain different data types

💡 **Why use tuples?** Their immutability makes them ideal for fixed collections of related data, ensuring data integrity and allowing use as dictionary keys.



</> Creating Tuples

```
my_tuple = (1, "hello", 3.14)
another_tuple = 10, 20, "Python"
single_element = ("item",)
```

🔍 Accessing Elements

```
my_tuple[0] # Returns 10
my_tuple[-1] # Returns 3.14
my_tuple[1:3] # Returns ("hello", 3.14)
```

⚠️ Immutability

```
my_tuple = (1, 2, 3)
# my_tuple[0] = 5 # TypeError!
```

Tuple Creation and When to Use

Tuple Creation

Tuples are created using parentheses ():

```
my_tuple = (1, "hello", 3.14, True)
```

Single-element tuple requires trailing comma:

```
single_element_tuple = ("item",)
```

Accessing Elements

```
my_tuple = (10, 20, 30, 40, 50)
print(my_tuple[0]) # First element: 10
print(my_tuple[-1]) # Last element: 50
print(my_tuple[1:4]) # Slicing: (20, 30, 40)
```

 Tuples support indexing and slicing like lists, but cannot be modified.

When to Use Tuples

Dictionary Keys

```
coordinates_data = {
    (40.7128, -74.0060): "New York City",
    (34.0522, -118.2437): "Los Angeles"
}
```

Performance

Tuples are more memory-efficient and faster than lists for lists for fixed collections.

Function Returns

```
def get_user_info():
    return "Alice", 30, "Engineer"

name, age, profession = get_user_info()
```

Sets: The Unique Collection

A Python set is an **unordered, mutable collection** of unique elements.

Sets automatically eliminate duplicate values.

Unordered

Elements have no defined order

Unique Elements

Discards duplicate values

Mutable

Elements can be added/removed

Non-indexable

Cannot be accessed by index

Set Creation & Properties

```
my_set = {1, 2, 3, 2, 4}
# Output: {1, 2, 3, 4}
fruits = {"apple", "banana", "cherry", "apple"}
```

Visual Representation:



Original: [1, 2, 3, 2, 4]

Duplicates removed: {1, 2, 3, 4}

Note: To create an empty set, use `set()`, not `{}`.

Set Operations and Methods

Creating Sets

Using curly braces:

```
my_set = {1, 2, 3, 2, 4}
print(my_set) # Output: {1, 2, 3, 4}
```

Using set() constructor:

```
empty_set = set()
list_to_set = set([1, 2, 1, 3])
```

Adding & Removing Elements

Adding elements:

```
fruits = {"apple", "banana"}
fruits.add("cherry")
print(fruits) # Output: {"apple", "banana",
"cherry"}
```

 **Note:** Removing non-existent element raises KeyError.

Set Operations

Union

All elements from both sets

```
set_a | set_b
# or
set_a.union(set_b)

{1, 2, 3} | {3, 4, 5} = {1, 2, 3, 4, 5}
```

Intersection

Common elements in both sets

```
set_a & set_b
# or
set_a.intersection(set_b)

{1, 2, 3} & {3, 4, 5} = {3}
```

Difference

Elements in first set but not second

```
set_a - set_b
# or
set_a.difference(set_b)

{1, 2, 3} - {3, 4, 5} = {1, 2}
```

Symmetric Difference

Elements in either set, not both

```
set_a ^ set_b
# or
set_a.symmetric_difference(set_b)

{1, 2, 3} ^ {3, 4, 5} = {1, 2, 4, 5}
```

 **Tip:** Use the `in` operator to check element existence.

Dictionaries: The Key-Value Store

Definition

A dictionary in Python is a mutable collection of key-value pairs where each unique key maps to a specific value.

Key Characteristics

- ✓ Mutability: Can be modified after creation
- ✓ Unique keys: Each key can appear only once
- ✓ Insertion order: Maintains order since Python 3.7+
- ✓ Efficient retrieval: Fast access to values via keys

Dictionary Structure

```
# Creating a dictionary
student = {"name": "Alice", "age": 20, "major": "Computer Science"}
```

Keys

Unique identifiers

Values

Associated data

Common Operations

```
# Accessing values
student["name"] # "Alice"
```

```
# Removing items
del student["major"]
```

```
# Adding/modifying
student["gpa"] = 3.8
```

```
# Checking keys
"name" in student # True
```

Dictionary Methods and Iteration

Safe Value Access with .get()

The `.get()` method retrieves a value for a given key, preventing `KeyError` if the key doesn't exist.

```
# Using .get() for an existing key
student = {"name": "Bob", "id": 101}
print(student.get("name")) # Output: Bob
```

```
# Using .get() for a non-existent key
print(student.get("email")) # Output: None
```

```
# Using .get() with a default value
print(student.get("phone", "N/A")) # Output: N/A
```

Dictionary Iteration



.keys()

Iterates through keys



.values()

Iterates through values



.items()

Iterates through key-value pairs

```
# Iterating through keys
grades = {"math": 90, "science": 85}
for subject in grades.keys():
    print(subject)
```

```
# Iterating through key-value pairs
for subject, score in grades.items():
    print(f"{subject}: {score}")
```

Data Structure Comparison

Python offers four core data structures with distinct characteristics. Here's a comparison of their key features:

Feature	 List	 Tuple	 Set	 Dictionary
Mutability	Mutable (changeable)	Immutable (unchangeable)	Mutable (elements can be added/removed)	Mutable (values can be changed, keys are immutable)
Ordering	Ordered (maintains insertion order)			Ordered (Python 3.7+ maintains insertion order)
Duplicates	Allows duplicate elements	Allows duplicate elements	Does not allow duplicate elements	Keys must be unique; values can be duplicated
Syntax	[item1, item2]			

Choosing the Right Data Structure

Selecting the appropriate data structure is crucial for efficient Python code. Consider these guidelines based on your needs:



Use Lists When:

- ✓ You need an ordered collection that can be modified after creation
- ✓ Implementing dynamic collections, stacks, or queues
- ✓ Order of elements matters for your algorithm



Use Tuples When:

- ✓ You need an ordered collection that should not be changed
- ✓ Representing fixed collections of related data (coordinates)
- ✓ Using as dictionary keys (if all elements are immutable)



Use Sets When:

- ✓ You need a collection of unique items
- ✓ Order of elements is not important
- ✓ Performing mathematical set operations or membership testing



Use Dictionaries When:

- ✓ Storing data as key-value pairs for fast lookups
- ✓ Representing structured data (similar to JSON objects)
- ✓ Creating lookup tables or counting frequency of items

Practice Exercises

Exercise 1: List Manipulation

Remove element at index 4 from the list, then insert it at the 2nd position and add it to the end.

```
my_list = [54, 44, 27, 79, 91, 41]
```

Expected Output: [54, 44, 91, 27, 79, 41, 91]

Exercise 3: Dictionary Value Extraction

Given a dictionary, extract all unique values and store them in a list.

```
data_speed = {'jan': 47, 'feb': 52, 'march': 47, 'April': 44, 'May': 52, 'June': 53, 'july': 54, 'Aug': 44, 'Sept': 54}
```

Expected Output: Unique values: [47, 52, 44, 53, 54]

Exercise 2: Set Operations

Find the common elements between two sets and remove them from the first set.

```
set_a = {23, 42, 65, 57, 78, 83, 29}
```

Expected Output: Intersection {57, 83, 29}, First Set after removal {65, 42, 78, 23}

Exercise 4: List to Tuple Conversion

Remove duplicates from a list, convert to a tuple, and find the min and max values.

```
numbers_list = [87, 45, 41, 65, 94, 41, 99, 94]
```

Expected Output: Unique items: [87, 45, 41, 65, 99], Tuple: (87, 45, 41, 65, 99), Min: 41, Max: 99